

FA6_QUIJANO

Julian Philip S. Quijano

2026-05-01

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.4.3
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
## Warning: package 'tibble' was built under R version 4.4.3
```

```
## Warning: package 'tidyr' was built under R version 4.4.3
```

```
## Warning: package 'readr' was built under R version 4.4.3
```

```
## Warning: package 'purrr' was built under R version 4.4.3
```

```
## Warning: package 'dplyr' was built under R version 4.4.3
```

```
## Warning: package 'forcats' was built under R version 4.4.3
```

```
## Warning: package 'lubridate' was built under R version 4.4.3
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.4      v readr      2.1.5
```

```
## v forcats    1.0.1      v stringr   1.5.1
```

```
## v ggplot2    4.0.1      v tibble    3.2.1
```

```
## v lubridate  1.9.4      v tidyr     1.3.1
```

```
## v purrr      1.0.4
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.4.3
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```
library(nnet)
library(MLmetrics)
```

```
## Warning: package 'MLmetrics' was built under R version 4.4.3
```

```
##
## Attaching package: 'MLmetrics'
##
## The following objects are masked from 'package:caret':
##
##     MAE, RMSE
##
## The following object is masked from 'package:base':
##
##     Recall
```

```
data <- read.csv("customer_segmentation.csv")
```

```
colnames(data) <- c(
  "CustomerID", "Age", "Income", "Gender",
  "Category", "AvgSpend", "Visits", "Segment"
)
```

```
str(data)
```

```
## 'data.frame':    10532 obs. of  8 variables:
## $ CustomerID: int  1 2 3 4 5 6 7 8 9 10 ...
## $ Age       : int  56 69 46 32 60 25 38 56 36 40 ...
## $ Income    : int  106 66 110 50 73 48 100 131 37 106 ...
## $ Gender    : chr  "Female" "Female" "Male" "Male" ...
## $ Category  : chr  "Fashion" "Home" "Fashion" "Electronics" ...
## $ AvgSpend  : num  163 163 105 110 142 ...
## $ Visits    : int  16 31 29 26 38 22 20 33 34 34 ...
## $ Segment   : chr  "Premium Shopper" "Budget Shopper" "Budget Shopper" "Regular Shopper" ...
```

```
summary(data)
```

```
##      CustomerID      Age      Income      Gender
## Min.   :    1  Min.   :18.00  Min.   : 30.00  Length:10532
## 1st Qu.: 2634  1st Qu.:31.00  1st Qu.: 59.00  Class :character
## Median : 5266  Median :43.00  Median : 89.00  Mode  :character
## Mean   : 5266  Mean   :43.59  Mean   : 89.18
## 3rd Qu.: 7899  3rd Qu.:56.00  3rd Qu.:118.00
```

```
## Max. :10532 Max. :69.00 Max. :149.00
## Category AvgSpend Visits Segment
## Length:10532 Min. : 10.00 Min. : 5.00 Length:10532
## Class :character 1st Qu.: 56.71 1st Qu.:13.00 Class :character
## Mode :character Median :104.69 Median :22.00 Mode :character
## Mean :104.30 Mean :21.92
## 3rd Qu.:150.89 3rd Qu.:31.00
## Max. :199.96 Max. :39.00
```

```
head(data)
```

```
## CustomerID Age Income Gender Category AvgSpend Visits Segment
## 1 1 56 106 Female Fashion 163.4528 16 Premium Shopper
## 2 2 69 66 Female Home 163.0205 31 Budget Shopper
## 3 3 46 110 Male Fashion 104.5413 29 Budget Shopper
## 4 4 32 50 Male Electronics 110.0646 26 Regular Shopper
## 5 5 60 73 Female Others 142.2546 38 Regular Shopper
## 6 6 25 48 Male Home 106.7621 22 Budget Shopper
```

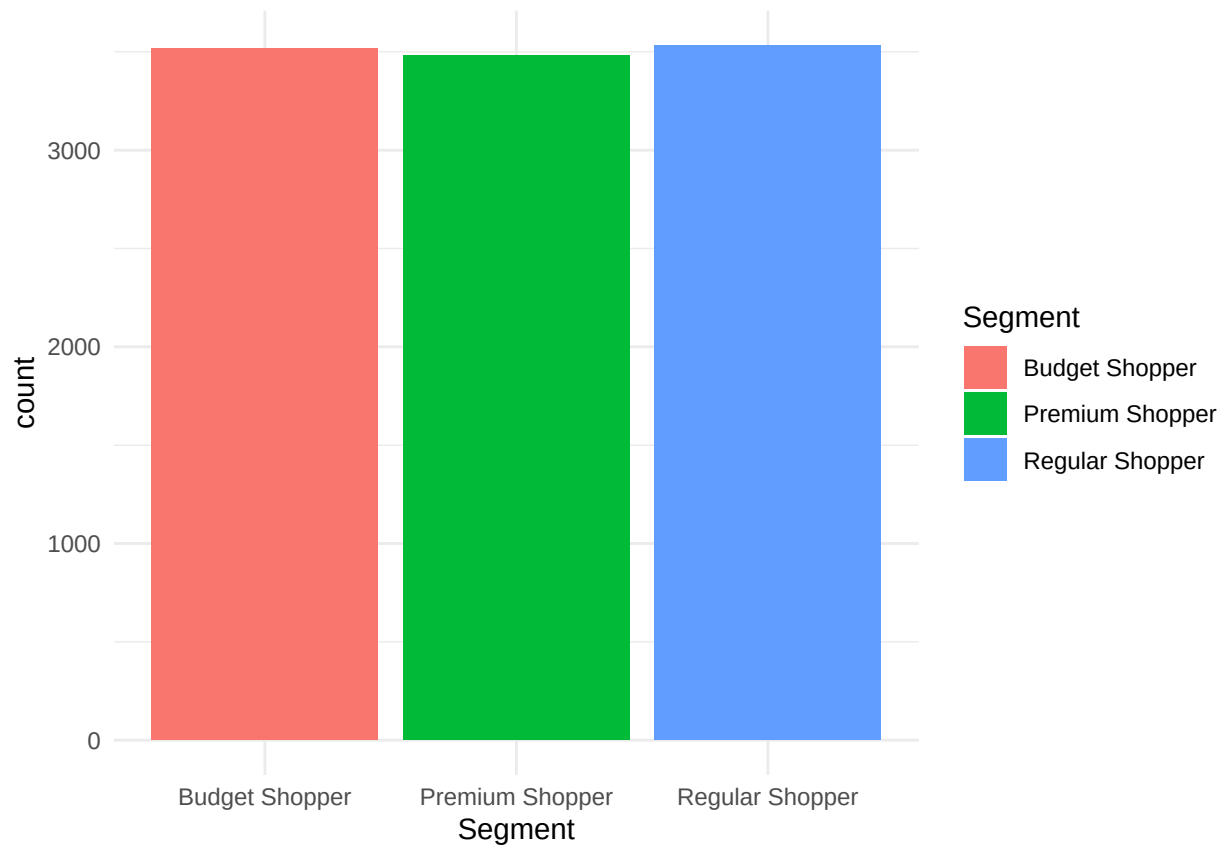
```
colSums(is.na(data))
```

```
## CustomerID Age Income Gender Category AvgSpend Visits
## 0 0 0 0 0 0 0
## Segment
## 0
```

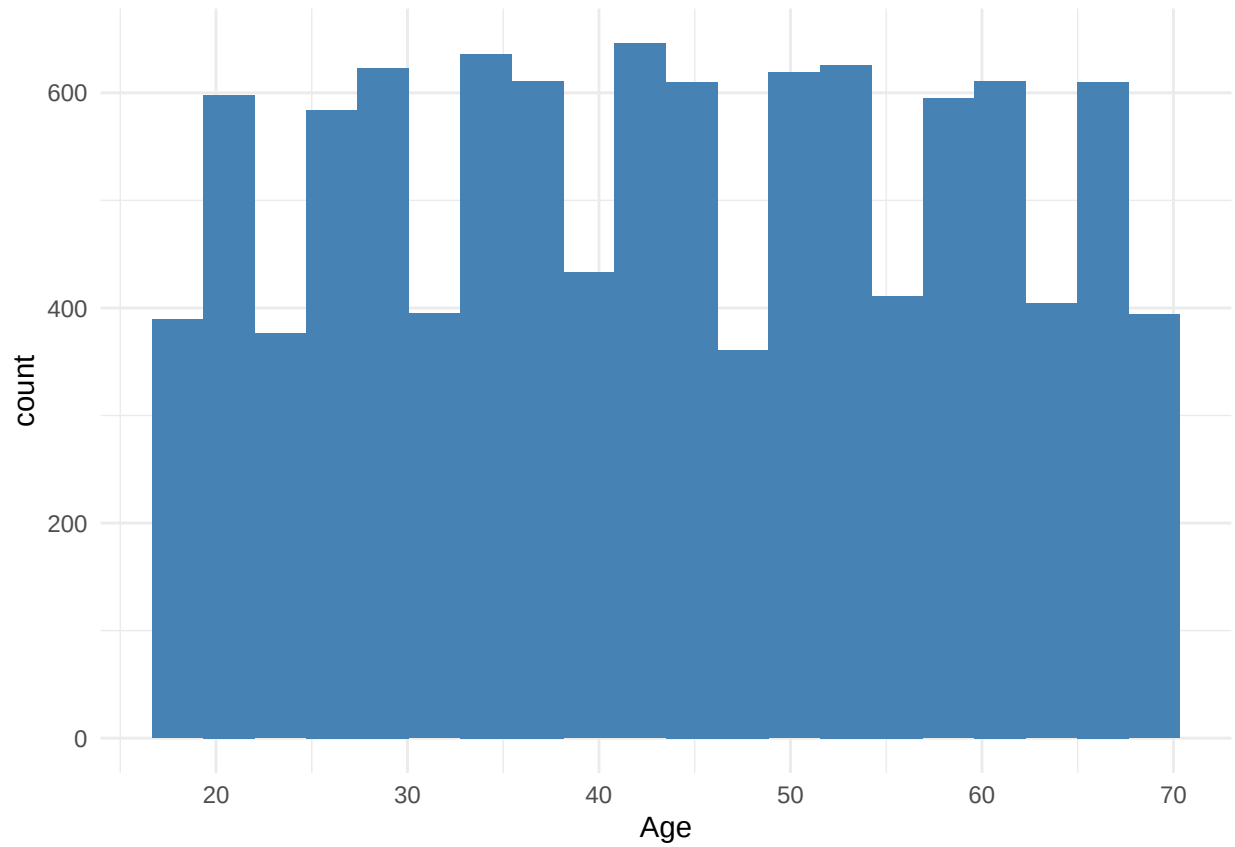
```
table(data$Segment)
```

```
##
## Budget Shopper Premium Shopper Regular Shopper
## 3516 3483 3533
```

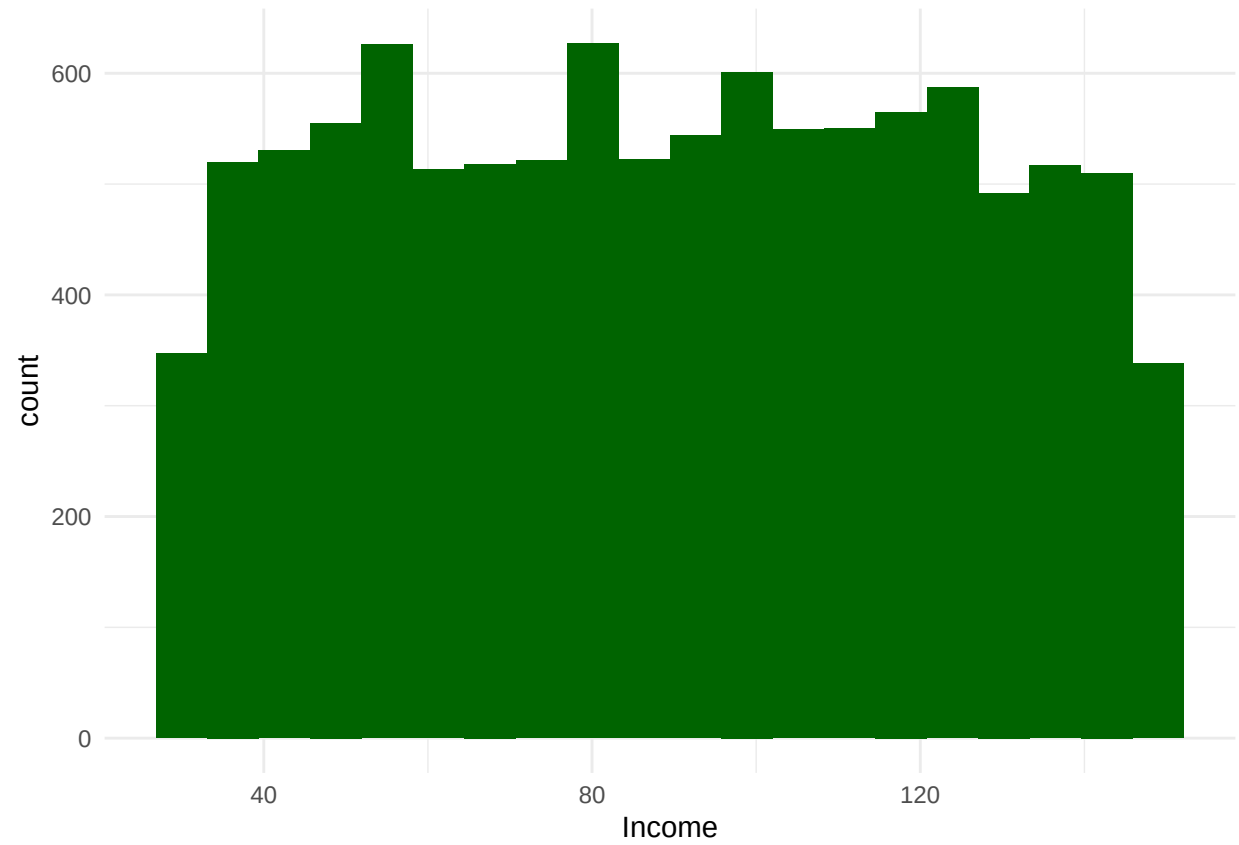
```
ggplot(data, aes(x = Segment, fill = Segment)) +
  geom_bar() +
  theme_minimal()
```



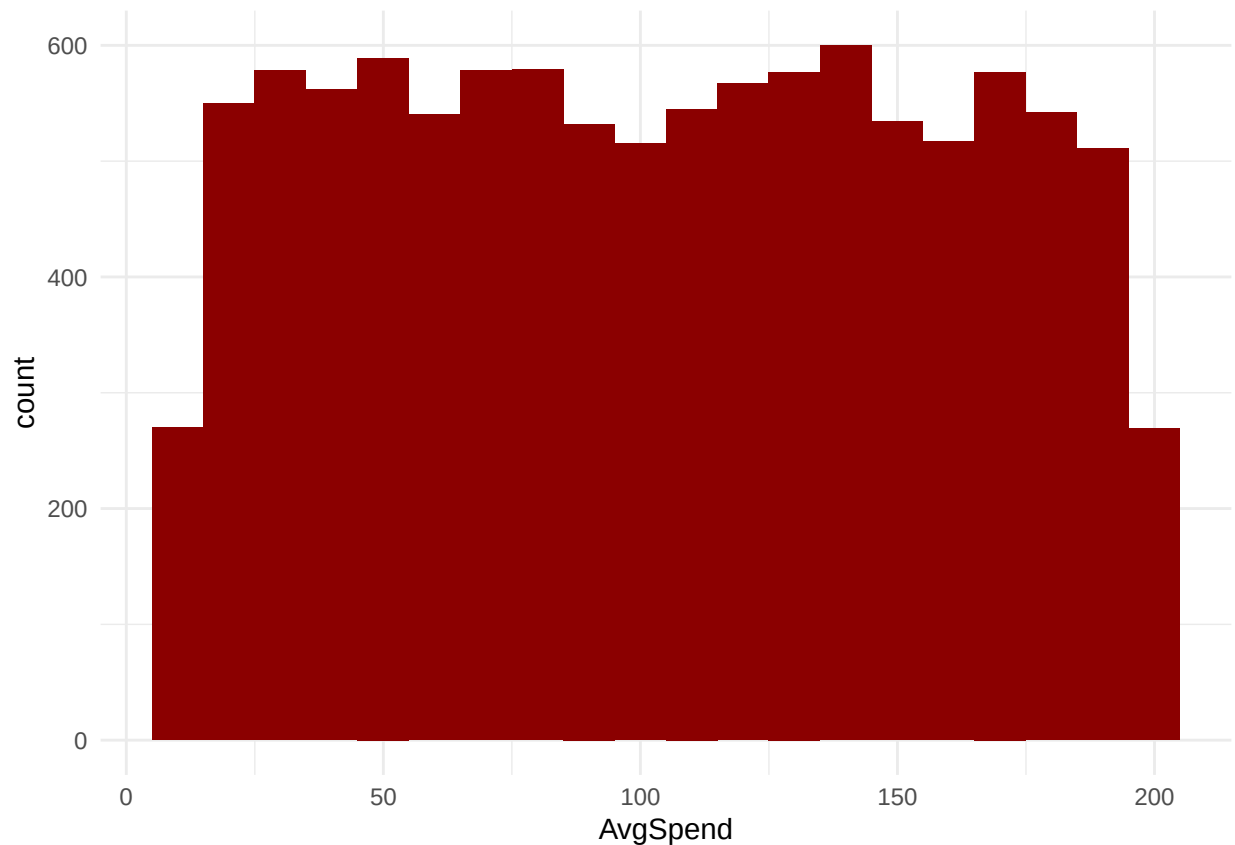
```
ggplot(data, aes(Age)) +  
  geom_histogram(bins = 20, fill = "steelblue") +  
  theme_minimal()
```



```
ggplot(data, aes(Income)) +  
  geom_histogram(bins = 20, fill = "darkgreen") +  
  theme_minimal()
```



```
ggplot(data, aes(AvgSpend)) +  
  geom_histogram(bins = 20, fill = "darkred") +  
  theme_minimal()
```



```
data$Gender <- as.factor(data$Gender)
data$Category <- as.factor(data$Category)
data$Segment <- as.factor(data$Segment)
```

```
dummies <- dummyVars(Segment ~ ., data = data)
data_encoded <- data.frame(predict(dummies, newdata = data))
```

```
## Warning in model.frame.default(Terms, newdata, na.action = na.action, xlev =
## object$lvls): variable 'Segment' is not a factor
```

```
# Add target back
data_encoded$Segment <- data$Segment

preProc <- preProcess(
  data_encoded[, !colnames(data_encoded) %in% c("Segment")],
  method = c("center", "scale")
)

data_scaled <- predict(preProc, data_encoded)

data_final <- data_scaled
```

```
set.seed(123)
```

```
trainIndex <- createDataPartition(data_final$Segment, p = 0.8, list = FALSE)
```

```
trainData <- data_final[trainIndex, ]  
testData <- data_final[-trainIndex, ]
```

```
model <- multinom(Segment ~ ., data = trainData)
```

```
## # weights: 42 (26 variable)  
## initial value 9258.005757  
## iter 10 value 9249.312613  
## final value 9248.931436  
## converged
```

```
summary(model)
```

```
## Call:  
## multinom(formula = Segment ~ ., data = trainData)  
##  
## Coefficients:  
##              (Intercept)  CustomerID      Age      Income Gender.Female  
## Premium Shopper -0.008278521 -0.008300029 0.01618791 -0.02797314  0.009293633  
## Regular Shopper  0.005536995 -0.009636473 0.01030734 -0.03985586  0.015477042  
##              Gender.Male Category.Books Category.Electronics  
## Premium Shopper -0.009293633   -0.04987944           -0.003615062  
## Regular Shopper -0.015477042   -0.05095139           0.003181358  
##              Category.Fashion Category.Home Category.Others  AvgSpend  
## Premium Shopper      0.04746146  0.0049675343      0.002056604 -0.01394197  
## Regular Shopper      0.02469683 -0.0002329911      0.023408927 -0.03675482  
##              Visits  
## Premium Shopper -0.020591574  
## Regular Shopper -0.008428884  
##  
## Std. Errors:  
##              (Intercept) CustomerID      Age      Income Gender.Female  
## Premium Shopper  0.02675714 0.02676737 0.02670701 0.02680089  0.01338083  
## Regular Shopper  0.02666747 0.02667770 0.02661532 0.02670943  0.01333494  
##              Gender.Male Category.Books Category.Electronics  
## Premium Shopper  0.01338083   0.02125471           0.02151402  
## Regular Shopper  0.01333494   0.02119369           0.02137324  
##              Category.Fashion Category.Home Category.Others  AvgSpend  
## Premium Shopper      0.02165452  0.02138267      0.02123167 0.02676739  
## Regular Shopper      0.02181526  0.02136143      0.02096701 0.02668233  
##              Visits  
## Premium Shopper 0.02688026  
## Regular Shopper 0.02679077  
##  
## Residual Deviance: 18497.86  
## AIC: 18541.86
```

```
control <- trainControl(method = "cv", number = 5)
```

```
tuned_model <- train(
```



```

Segment ~ .,
data = trainData,
method = "multinom",
trControl = control,
tuneLength = 5
)

```

```

## # weights: 42 (26 variable)
## initial value 7405.745438
## iter 10 value 7399.032568
## iter 20 value 7398.712448
## final value 7398.679770
## converged
## # weights: 42 (26 variable)
## initial value 7405.745438
## iter 10 value 7399.033644
## iter 20 value 7398.714041
## final value 7398.681166
## converged
## # weights: 42 (26 variable)
## initial value 7405.745438
## iter 10 value 7399.032676
## iter 20 value 7398.712607
## final value 7398.679909
## converged
## # weights: 42 (26 variable)
## initial value 7405.745438
## iter 10 value 7399.032579
## iter 20 value 7398.712464
## final value 7398.679784
## converged
## # weights: 42 (26 variable)
## initial value 7405.745438
## iter 10 value 7399.032569
## iter 20 value 7398.712449
## final value 7398.679771
## converged
## # weights: 42 (26 variable)
## initial value 7407.942662
## iter 10 value 7400.534738
## iter 20 value 7399.747506
## final value 7399.666248
## converged
## # weights: 42 (26 variable)
## initial value 7407.942662
## iter 10 value 7400.536668
## iter 20 value 7399.750355
## final value 7399.667662
## converged
## # weights: 42 (26 variable)
## initial value 7407.942662
## iter 10 value 7400.534931
## iter 20 value 7399.747790

```

```

## final value 7399.666390
## converged
## # weights: 42 (26 variable)
## initial value 7407.942662
## iter 10 value 7400.534757
## iter 20 value 7399.747534
## final value 7399.666262
## converged
## # weights: 42 (26 variable)
## initial value 7407.942662
## iter 10 value 7400.534739
## iter 20 value 7399.747509
## final value 7399.666250
## converged
## # weights: 42 (26 variable)
## initial value 7405.745438
## iter 10 value 7395.322678
## iter 20 value 7394.885754
## iter 20 value 7394.885699
## final value 7394.884795
## converged
## # weights: 42 (26 variable)
## initial value 7405.745438
## iter 10 value 7395.323968
## iter 20 value 7394.887918
## iter 20 value 7394.887862
## final value 7394.886964
## converged
## # weights: 42 (26 variable)
## initial value 7405.745438
## iter 10 value 7395.322807
## iter 20 value 7394.885971
## iter 20 value 7394.885915
## final value 7394.885012
## converged
## # weights: 42 (26 variable)
## initial value 7405.745438
## iter 10 value 7395.322691
## iter 20 value 7394.885776
## iter 20 value 7394.885720
## final value 7394.884817
## converged
## # weights: 42 (26 variable)
## initial value 7405.745438
## iter 10 value 7395.322680
## iter 20 value 7394.885756
## iter 20 value 7394.885701
## final value 7394.884797
## converged
## # weights: 42 (26 variable)
## initial value 7406.844050
## iter 10 value 7397.316083
## iter 20 value 7396.884992
## final value 7396.821207

```

```

## converged
## # weights: 42 (26 variable)
## initial value 7406.844050
## iter 10 value 7397.317552
## iter 20 value 7396.887233
## final value 7396.823064
## converged
## # weights: 42 (26 variable)
## initial value 7406.844050
## iter 10 value 7397.316230
## iter 20 value 7396.885216
## final value 7396.821393
## converged
## # weights: 42 (26 variable)
## initial value 7406.844050
## iter 10 value 7397.316098
## iter 20 value 7396.885014
## final value 7396.821226
## converged
## # weights: 42 (26 variable)
## initial value 7406.844050
## iter 10 value 7397.316085
## iter 20 value 7396.884994
## final value 7396.821209
## converged
## # weights: 42 (26 variable)
## initial value 7405.745438
## iter 10 value 7398.503657
## iter 20 value 7398.041196
## final value 7398.002969
## converged
## # weights: 42 (26 variable)
## initial value 7405.745438
## iter 10 value 7398.505227
## iter 20 value 7398.042429
## final value 7398.004397
## converged
## # weights: 42 (26 variable)
## initial value 7405.745438
## iter 10 value 7398.503814
## iter 20 value 7398.041319
## final value 7398.003112
## converged
## # weights: 42 (26 variable)
## initial value 7405.745438
## iter 10 value 7398.503673
## iter 20 value 7398.041208
## final value 7398.002983
## converged
## # weights: 42 (26 variable)
## initial value 7405.745438
## iter 10 value 7398.503659
## iter 20 value 7398.041197
## final value 7398.002970

```

```
## converged
## # weights: 42 (26 variable)
## initial value 9258.005757
## iter 10 value 9249.312624
## final value 9248.931450
## converged
```

```
print(tuned_model)
```

```
## Penalized Multinomial Regression
##
## 8427 samples
## 12 predictor
## 3 classes: 'Budget Shopper', 'Premium Shopper', 'Regular Shopper'
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 6741, 6743, 6741, 6742, 6741
## Resampling results across tuning parameters:
##
## decay Accuracy Kappa
## 0e+00 0.3340428 0.0004337209
## 1e-04 0.3341614 0.0006124922
## 1e-03 0.3342800 0.0007897974
## 1e-02 0.3341614 0.0006110261
## 1e-01 0.3340428 0.0004337209
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was decay = 0.001.
```

```
predictions <- predict(model, testData)
probabilities <- predict(model, testData, type = "prob")
```

```
cm <- confusionMatrix(predictions, testData$Segment)
print(cm)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction    Budget Shopper Premium Shopper Regular Shopper
## Budget Shopper          304           273           267
## Premium Shopper         146           152           167
## Regular Shopper         253           271           272
##
## Overall Statistics
##
##              Accuracy : 0.3458
##              95% CI : (0.3255, 0.3666)
##              No Information Rate : 0.3354
##              P-Value [Acc > NIR] : 0.1605
##
##              Kappa : 0.0181
```

```
##
## McNemar's Test P-Value : 1.017e-13
##
## Statistics by Class:
##
##           Class: Budget Shopper Class: Premium Shopper
## Sensitivity           0.4324           0.21839
## Specificity           0.6148           0.77786
## Pos Pred Value        0.3602           0.32688
## Neg Pred Value        0.6836           0.66829
## Prevalence            0.3340           0.33064
## Detection Rate        0.1444           0.07221
## Detection Prevalence  0.4010           0.22090
## Balanced Accuracy     0.5236           0.49812
##
##           Class: Regular Shopper
## Sensitivity           0.3853
## Specificity           0.6254
## Pos Pred Value        0.3417
## Neg Pred Value        0.6684
## Prevalence            0.3354
## Detection Rate        0.1292
## Detection Prevalence  0.3781
## Balanced Accuracy     0.5054
```

```
LogLoss(probabilities, testData$Segment)
```

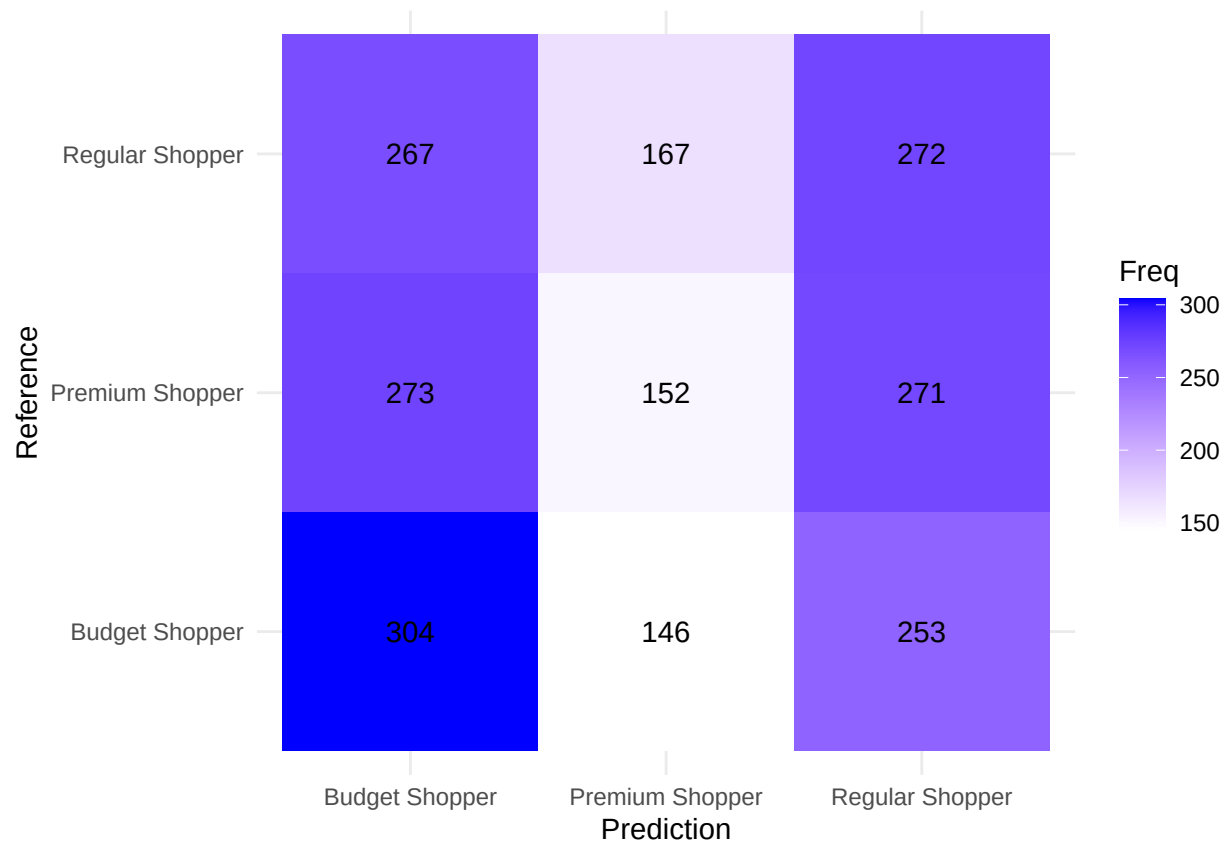
```
## Warning in Ops.factor(y_true, log(y_pred)): '*' not meaningful for factors
```

```
## Warning in Ops.factor(1, y_true): '-' not meaningful for factors
```

```
## [1] NA
```

```
cm_table <- as.data.frame(cm$table)

ggplot(cm_table, aes(Prediction, Reference, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = Freq)) +
  scale_fill_gradient(low = "white", high = "blue") +
  theme_minimal()
```



```
coef(model)
```

```
##              (Intercept)  CustomerID      Age      Income Gender.Female
## Premium Shopper -0.008278521 -0.008300029 0.01618791 -0.02797314  0.009293633
## Regular Shopper  0.005536995 -0.009636473 0.01030734 -0.03985586  0.015477042
##              Gender.Male Category.Books Category.Electronics
## Premium Shopper -0.009293633    -0.04987944          -0.003615062
## Regular Shopper -0.015477042    -0.05095139           0.003181358
##              Category.Fashion Category.Home Category.Others  AvgSpend
## Premium Shopper      0.04746146  0.0049675343      0.002056604 -0.01394197
## Regular Shopper      0.02469683 -0.0002329911      0.023408927 -0.03675482
##              Visits
## Premium Shopper -0.020591574
## Regular Shopper -0.008428884
```

```
data$Income_Age <- data$Income * data$Age

dummies <- dummyVars(Segment ~ ., data = data)
data_encoded <- data.frame(predict(dummies, newdata = data))
```

```
## Warning in model.frame.default(Terms, newdata, na.action = na.action, xlev =
## object$lvls): variable 'Segment' is not a factor
```

```

data_encoded$Segment <- data$Segment

preProc <- preProcess(
  data_encoded[, !colnames(data_encoded) %in% c("Segment")],
  method = c("center", "scale")
)

data_scaled <- predict(preProc, data_encoded)
data_final <- data_scaled

preProc <- preProcess(
  data_encoded[, !colnames(data_encoded) %in% c("Segment")],
  method = c("center", "scale")
)

data_scaled <- predict(preProc, data_encoded)
data_final <- data_scaled

model <- multinom(Segment ~ ., data = trainData)

```

```

## # weights: 42 (26 variable)
## initial value 9258.005757
## iter 10 value 9249.312613
## final value 9248.931436
## converged

```

```
coef(model)
```

```

##              (Intercept)  CustomerID      Age      Income Gender.Female
## Premium Shopper -0.008278521 -0.008300029 0.01618791 -0.02797314 0.009293633
## Regular Shopper 0.005536995 -0.009636473 0.01030734 -0.03985586 0.015477042
##              Gender.Male Category.Books Category.Electronics
## Premium Shopper -0.009293633 -0.04987944 -0.003615062
## Regular Shopper -0.015477042 -0.05095139 0.003181358
##              Category.Fashion Category.Home Category.Others AvgSpend
## Premium Shopper 0.04746146 0.0049675343 0.002056604 -0.01394197
## Regular Shopper 0.02469683 -0.0002329911 0.023408927 -0.03675482
##              Visits
## Premium Shopper -0.020591574
## Regular Shopper -0.008428884

```

A multinomial logistic regression model was developed to classify customers into Budget, Regular, and Premium segments based on demographic and behavioral features. The dataset was first preprocessed by encoding categorical variables (gender and product category) and standardizing numerical variables such as age, income, and average spending. The data was then split into training and testing sets (80/20). The model was trained on the training set and evaluated on the test set using metrics including accuracy, confusion matrix, precision, recall, and log-loss.

The model achieved an accuracy of 34.58%, which is only slightly above the baseline probability of random guessing. The confusion matrix revealed that the model struggled to correctly distinguish between segments, particularly for Premium customers, which had low sensitivity. Among the features, Average Spend per Visit and Annual Income appeared to have the strongest influence on predictions, as they are directly related to

customer purchasing behavior. However, the overall weak performance suggests that these relationships are not sufficiently captured by a linear model.

To improve segmentation, more advanced models such as decision trees or ensemble methods (e.g., random forests or gradient boosting) should be considered, as they can capture nonlinear relationships. Additionally, feature engineering—such as creating interaction terms or incorporating more detailed behavioral data—may enhance predictive performance. From a business perspective, targeting strategies could still leverage spending behavior, with high spenders prioritized for premium offers and lower spenders targeted with discounts, but more accurate segmentation would require improved modeling techniques.